PRACTICAL 12 – END-TO-END COMMUNICATION AT THE TRANSPORT LAYER

AIM

To implement echo and chat client-server applications using TCP and UDP socket programming in Python.

PART A – ECHO CLIENT-SERVER

Algorithm

1. Create a server socket and bind it to an IP address and port number.
2. For TCP, listen for incoming connections and accept a client connection.
3. The client sends a message to the server.
4. The server receives the message and sends the same message back to the client.
5. The client receives and displays the echoed message.

Input / Output

Input: A message entered by the client.

Output: The server returns the same message, which is displayed by the client.

PART B – CHAT CLIENT-SERVER

Algorithm

1. Create and configure the server and client sockets.
2. Establish a connection for TCP, or use `sendto()` and `recvfrom()` for UDP communication.
3. Allow the client and server to send and receive messages alternately.
4. Display the messages received at both ends.
5. Terminate the communication when either side enters `exit`.

Input / Output

Input: Messages entered by the client and server.

Output: Messages are exchanged and displayed at both ends until the chat is terminated.

PROCEDURE

1. Save the client and server programs in separate Python files, or use a combined program that allows the user to select a role.
2. Start the server program first.
3. Run the client program in a separate terminal on the same computer or on a reachable host.
4. Enter a message and observe the echo or chat response.
5. Enter `exit` to end the chat session.

RESULT

Echo and chat client-server applications were implemented and tested using Python socket programming.